

1 Dòng CNN: gỡ từng nút thắt (The CNN Lineage)

PHỤ THUỘC C/12-co-che-huan-luyen (đặc biệt mục 2 — kết nối tắt là thành viên thứ tư của họ “giữ tích Jacobian gần 1”); B/11-gioi-han-tinh-toan (FLOPs, cường độ số học, chế độ nghẽn bộ nhớ).

TÓM TẮT

Mục này đọc mười lăm năm lịch sử *CNN* như một chuỗi gỡ nút thắt: mỗi kiến trúc ra đời để phá đúng một rào cản của kiến trúc trước, và tạo ra rào cản mới cho kiến trúc sau. Đọc xong bạn nhìn một *backbone* lạ và gọi tên được nó đang tối ưu cho ràng buộc nào: tham số, độ sâu, FLOPs, hay độ trễ thật. Nhờ đó bạn chọn backbone theo ràng buộc của mình, thay vì theo bảng xếp hạng. Nếu chỉ nhớ một điều: bài học lớn nhất của cả dòng nằm ở mắt xích cuối, ConvNeXt. Rất nhiều “cải tiến kiến trúc” trong lịch sử hoá ra là cải tiến *công thức huấn luyện*. Nên mọi so sánh kiến trúc không kiểm soát recipe đều đáng ngờ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **convolution** — phép nhân một cửa sổ trọng số nhỏ, gọi là kernel hay filter, trượt khắp ảnh; vì cùng bộ trọng số dùng cho mọi vị trí nên số tham số không phụ thuộc kích thước ảnh.
- **kênh (channel)** — chiều thứ ba của một bản đồ đặc trưng — ảnh RGB có 3 kênh đầu vào, còn một tầng giữa mạng có thể có 256 kênh, mỗi kênh là một loại đặc trưng mà mạng học được.
- **stride** — bước nhảy của cửa sổ khi trượt; stride 2 làm bản đồ đặc trưng nhỏ đi một nửa mỗi chiều, nên nó là cách chính để giảm độ phân giải và tăng tầm nhìn.
- **trường tiếp nhận (receptive field)** — vùng pixel đầu vào ảnh hưởng tới một giá trị đầu ra; nó lớn dần theo chiều sâu, và cả mục này xoay quanh việc mở rộng nó cho rộng.
- **pooling** — phép gộp một vùng nhỏ thành một giá trị (lấy lớn nhất hoặc lấy trung bình), dùng để giảm độ phân giải mà không tốn tham số.
- **FLOPs** — số phép nhân-cộng mà mô hình thực hiện; đây là chỉ số đại diện phổ biến cho chi phí, nhưng *không* tỉ lệ với độ trễ thật.
- **cường độ số học (arithmetic intensity)** — số phép tính thực hiện trên mỗi byte dữ liệu đọc từ bộ nhớ; thấp thì kernel bị nghẽn ở bảng thông chữ không ở khả năng tính toán, và giảm FLOPs lúc đó gần như vô ích.
- **recipe (công thức huấn luyện)** — toàn bộ các lựa chọn không thuộc kiến trúc — optimizer, lịch học, số epoch, augmentation — và bài học cuối mục là chúng giải thích phần lớn chênh lệch giữa các kiến trúc.

A. Đặt vấn đề

Toàn bộ chương 13 là một cuộc tranh luận kéo dài về một câu hỏi: *nên cài sẵn bao nhiêu giả định vào mô hình?* *Convolution* là một câu trả lời rất mạnh cho câu hỏi đó, và nó cài sẵn đúng ba giả định. Thứ nhất là **tính cục bộ (locality)** — một pixel chỉ liên quan trực tiếp tới các pixel lân cận, nên trọng số bằng 0 ở ngoài cửa sổ $k \times k$. Thứ hai là **chia sẻ trọng số (weight sharing)**, tương đương với **đẳng biến tịnh tiến (translation equivariance)** — một mẫu hình có ý nghĩa như nhau dù nó nằm ở đâu trong ảnh. Thứ ba là **phân cấp (hierarchy)** — đặc trưng phức tạp được ghép từ đặc trưng đơn giản, nên tăng dần độ trừu tượng và giảm dần độ phân giải.

Ba giả định này đến từ vật lý của việc tạo ảnh chứ không từ toán học, và chúng đúng đủ tốt để một mạng conv cần ít dữ liệu hơn hẳn một mạng dày đặc cùng sức chứa. Nhưng chúng cũng khoá lại một không gian thiết kế, và lịch sử CNN chính là lịch sử của việc lần lượt phát hiện ra rào cản nào trong không gian đó đang chặn, rồi gỡ nó. Giá trị cụ thể của mục này: cầm một backbone lạ lên, bạn trả lời được ba câu mà không cần chạy thử. Nó đang tiết kiệm cái gì? Nó trả giá ở đâu? Và ràng buộc nào khiến người ta chọn nó?

TRỰC GIÁC

Đừng đọc dòng CNN như một cuộc thi xem ai chính xác hơn. Hãy đọc nó như lịch sử tối ưu một đường ống sản xuất, nơi mỗi thể hệ tìm ra *nút cổ chai hiện thời* rồi mở rộng đúng chỗ đó. Mở đúng nút cổ chai thì thông lượng tăng vọt, và nút cổ chai lập tức chuyển sang chỗ khác. Đó là lý do mỗi bước tiến trong bảng dưới đây lại sinh ra một vấn đề mới ở trực khác. Người đọc lịch sử theo cách này đoán được bước tiếp theo; người đọc theo cách “ai chính xác hơn” chỉ nhớ được tên.

B. Dòng thời gian: ba thời kỳ, ba nút thắt khác nhau

Nhìn từ xa, mười lăm năm này chia làm ba thời kỳ rõ rệt, và *nút thắt của mỗi thời kỳ nằm ở một trực khác nhau*. Thời kỳ đầu vướng ở tính khả thi. Thời kỳ giữa vướng ở khả năng tối ưu hoá. Thời kỳ sau vướng ở phần cứng và ở công thức huấn luyện.

Thời kỳ 1 — làm cho mạng sâu khả thi

nút thắt: tính toán và tối ưu hoá cơ bản



Thời kỳ 2 — đi sâu hơn và rẻ hơn

nút thắt: độ sâu không huấn luyện được, và số phép nhân



Thời kỳ 3 — hợp với phần cứng, và recipe

nút thắt: độ trễ thật, và công thức huấn luyện



Hình 1. Dòng CNN đặt trên trục thời gian và tách theo nút thắt được gỡ. Hai mắt xích được tô đậm vì chúng thay đổi cách ngành suy nghĩ chứ không chỉ thay đổi con số: ResNet cho thấy rào cản của mạng sâu là *tối ưu hoá* chứ không phải sức chứa, còn ConvNeXt cho thấy phần lớn khoảng cách giữa các kiến trúc là khoảng cách công thức huấn luyện.

Bảng 1 đọc cùng Hình 1 theo chiều dọc; cột cuối là cột đáng nhớ, vì nó cho thấy không bước nào miễn phí.

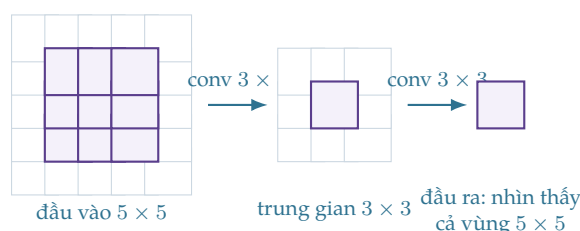
Bảng 1. Dòng CNN đọc theo mạch gỡ nút thắt. Cột cuối cho thấy mỗi bước tiến đẩy độ khó sang một trục khác chứ không xoá nó.

Bước	Gỡ nút thắt gì	Cái giá / vấn đề mới
LeNet	Chứng minh conv + pooling + huấn luyện gradient học được đặc trưng thị giác [1]	Chỉ chạy ở quy mô chữ số; chưa có dữ liệu và phần cứng để mở rộng
AlexNet	Chứng minh mạng sâu + GPU + ReLU + dropout là khả thi ở quy mô ImageNet [2]	Kiến trúc làm bằng tay, filter lớn 11×11 , rất nhiều tham số ở tầng FC
ZFNet	Trực quan hoá để thấy mạng học gì, từ đó chỉnh filter và stride	Đóng góp là phương pháp quan sát, không phải kiến trúc mới
VGG	Xếp chồng 3×3 : cùng trường tiếp nhận, ít tham số hơn, nhiều phi tuyến hơn [3]	Tầng FC chiếm phần lớn tham số; tổng FLOPs rất cao
Inception	Nhiều tỉ lệ song song; 1×1 làm nút thắt giảm chiều trước phép tính nặng [4], [5]	Kiến trúc phức tạp, nhiều siêu tham số nhánh, khó chỉnh và khó mở rộng
ResNet	Vấn đề <i>tối ưu hoá</i> của mạng sâu, bằng đường cộng đồng nhất [6]	Vẫn nhiều FLOPs; nhánh tắt buộc giữ thêm kích hoạt nên tốn bộ nhớ hơn
DenseNet	Tái dùng đặc trưng tối đa, ít tham số hơn ở cùng độ chính xác [7]	Phép nối làm bộ nhớ và băng thông phình ra nếu cài đặt ngây thơ
MobileNet	Chi phí tính toán trên thiết bị yếu, bằng depthwise separable [8]	Giảm FLOPs nhiều hơn giảm độ trễ thật, vì cường độ số học tụt
SENet	Mạng không tự điều chỉnh được tầm quan trọng giữa các kênh [9]	Thêm một điểm đồng bộ toàn cục; lợi ích nhỏ dần khi backbone đã mạnh
EfficientNet	Việc mở rộng mô hình bị làm tùy tiện theo một chiều [10]	Tối ưu cho FLOPs, không tối ưu cho một phần cứng cụ thể
ConvNeXt	Nghi vấn “ViT thắng nhờ kiến trúc”, bằng cách hiện đại hoá recipe của CNN [11]	Cho thấy phần lớn chênh lệch trước đó là do công thức huấn luyện, làm mọi so sánh kiến trúc cũ trở nên đáng ngờ

C. Ba phép tính tay giải thích ba bước quan trọng nhất

Ba bước có ảnh hưởng lớn nhất trong bảng đều rút gọn được thành một phép tính ngắn. Tự làm ba phép tính này một lần thì bạn nhớ chúng mãi.

VGG — vì sao xếp chồng 3×3 . Hai tầng 3×3 nối tiếp có cùng trường tiếp nhận với một tầng 5×5 ; ba tầng thì bằng 7×7 . Nhưng chi phí tham số khác hẳn. Với C kênh vào và ra, một tầng 5×5 tốn $25C^2$ tham số, còn hai tầng 3×3 chỉ tốn $2 \cdot 9C^2 = 18C^2$ — ít hơn 28 phần trăm. So 7×7 với ba tầng 3×3 thì là $49C^2$ đối với $27C^2$, ít hơn 45 phần trăm. Và ta còn được thêm hai lần phi tuyến. Nói gọn: cùng tầm nhìn, ít tham số hơn, nhiều khả năng biểu diễn hơn — một trong những phép đổi hiểm hơi tốt ở cả ba mặt.

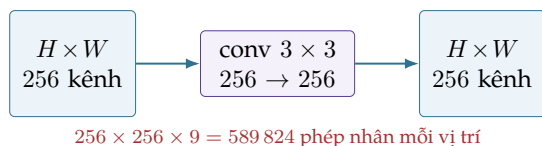


Hình 2. Hai tầng 3×3 nối tiếp cho một ô đầu ra nhìn thấy đúng vùng 5×5 của đầu vào. Cùng trường tiếp nhận với một tầng 5×5 , nhưng chỉ tốn $18C^2$ tham số thay vì $25C^2$. Và có thêm một lần phi tuyến ở giữa.

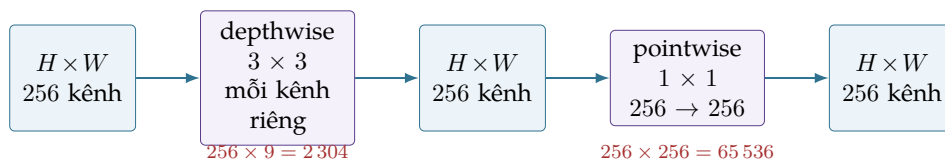
Hình 2 là toàn bộ lập luận của VGG dưới dạng hình. Điều nó *không* nói ra là nút thắt kế tiếp: tầng kết nối đầy đủ đầu tiên của VGG-16 có $7 \cdot 7 \cdot 512 \cdot 4096 \approx 103$ triệu tham số, tức khoảng ba phần tư toàn bộ mô hình nằm trong một tầng duy nhất không phải conv. Đó là lý do các thế hệ sau thay tầng FC bằng *global average pooling*.

Inception — vì sao 1×1 là nút thắt hiệu quả. Giả sử cần một conv 3×3 từ 256 kênh sang 256 kênh. Chi phí trực tiếp là $256 \cdot 256 \cdot 9 = 589\,824$ phép nhân-cộng cho mỗi vị trí không gian. Bây giờ chèn hai tầng 1×1 : giảm $256 \rightarrow 64$, làm 3×3 trong không gian 64 kênh, rồi mở lại $64 \rightarrow 256$. Tổng chi phí là $256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 = 16\,384 + 36\,864 + 16\,384 = 69\,632$, tức rẻ hơn 8,5 lần. Ý nghĩa của phép tính: phần đắt nhất của conv là tương tác giữa *mọi* kênh vào với *mọi* kênh ra, nhân với k^2 ; nếu ta làm phép tính không gian ở một không gian kênh hẹp thì cái k^2 chỉ nhân với một số nhỏ. Đây cũng chính là khối *bottleneck* của ResNet, và là ý tưởng nền của phần lớn kiến trúc hiệu quả về sau.

conv 3×3 thường: mọi kênh vào nói chuyện với mọi kênh ra, ở mọi vị trí



depthwise separable: tách phần không gian khỏi phần kênh



Nhưng bước depthwise đọc mỗi phần tử để làm vòn vẹn 9 phép nhân, nên cường độ số học của nó tụt xuống mức nghen bằng thông.

Hình 3. Vì sao depthwise separable rẻ, và vì sao “rẻ” theo FLOPs không có nghĩa là “nhanh”. Conv thường trả tiền cho tích của ba thứ cùng lúc: kênh vào, kênh ra, và diện tích cửa sổ. Phép tách chia nó thành hai bước, mỗi bước chỉ trả cho *hai* trong ba thứ đó — đây là toàn bộ nguồn gốc của tỉ số $1/C_{\text{out}} + 1/k^2$. Cái giá nằm ở dòng cuối: bước depthwise có rất ít phép tính trên mỗi byte đọc vào, nên nó rơi vào phía nghen bộ nhớ và thời gian thực tế không giảm theo tỉ lệ FLOPs.

Hình 3 bày ra cả hai vế của phép đổi trên cùng một sơ đồ, và đó là lý do nó đáng vẽ: con số 8,7 và lời cảnh báo về độ trễ luôn phải đọc cùng nhau.

MobileNet — và vì sao FLOPs không phải độ trễ. Depthwise separable tách một conv thành hai: một conv $k \times k$ chạy độc lập trên từng kênh, rồi một conv 1×1 trộn kênh. Tỉ số chi phí so với conv thường là $\frac{1}{C_{\text{out}}} + \frac{1}{k^2}$; với $k = 3$ và $C_{\text{out}} = 256$ thì bằng $0,0039 + 0,111 \approx 0,115$, tức rẻ hơn khoảng 8,7 lần. Nhưng độ trễ thực đo được hầu như không bao giờ giảm 8,7 lần, và lý do là thứ bạn đã gặp khi tối ưu CUDA: phần depthwise đọc mỗi phần tử đầu vào để làm vòn vẹn $k^2 = 9$ phép nhân, nên **cường độ số học** (*arithmetic intensity*) của nó cực thấp và nó rơi vào chế độ nghen bằng thông bộ nhớ. Giảm FLOPs của một kernel vốn đã nghen ở bộ nhớ thì gần như không giảm thời gian. Đây là ví dụ rõ nhất trong cả cây về việc một chỉ số đại diện — FLOPs — đo sai thứ mà ta quan tâm.

↪ **ràng buộc bởi B/11** — mô hình roofline nói chính xác khi nào giảm FLOPs không giảm thời gian: khi kernel đã nằm ở phía nghen bằng thông

D. ResNet: nút thắt là tối ưu hoá, không phải sức chứa

Đóng góp của ResNet hay bị kể sai thành “mạng sâu hơn thì mạnh hơn”. Bằng chứng gốc nói điều khác và mạnh hơn nhiều: một mạng phẳng 56 tầng có *sai số huấn luyện* cao hơn một mạng phẳng 20 tầng [6]. Sai số huấn luyện, không phải sai số kiểm thử. Nên đây không phải overfitting, và cũng không phải thiếu sức chứa. Mạng sâu hơn về nguyên tắc biểu diễn được mọi thứ mạng

nông biểu diễn được: chỉ cần các tầng thừa làm ánh xạ đồng nhất. Vấn đề là gradient descent không tìm ra nghiệm đó.

Kết nối tắt $y = x + F(x)$ làm hai việc cùng lúc, và cả hai đều đã xuất hiện ở chương 12. Về mặt biểu diễn, ánh xạ đồng nhất trở thành phương án *mặc định* — chỉ cần $F \approx 0$, điều mà khởi tạo nhỏ cho sẵn. Về mặt gradient, Jacobian của khối là $I + \partial F / \partial x$, nên tích các Jacobian qua nhiều khối luôn còn một thành phần bằng 1 không thể triệt tiêu; đây chính xác là thành viên thứ tư của họ kỹ thuật “giữ ρ gần 1” mà mục C/12/02 đã mở ngoặc để dành.

Khối phẳng



$\frac{\partial y}{\partial x} = J_2 J_1$ — mọi thừa số đều có thể nhỏ hơn 1
đường tắt: x đi thẳng, không qua phép toán nào

Khối residual



$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}$ — số hạng I không bao giờ triệt tiêu

Hệ quả kép: ánh xạ đồng nhất trở thành phương án *mặc định* (chỉ cần $F \approx 0$), và gradient luôn có một đường về không bị nhân dồn.

Hình 4. Kết nối tắt sửa vấn đề gì, nhìn từ phía lượt ngược. Ở khối phẳng, đạo hàm của đầu ra theo đầu vào là một tích thuần tuý, nên nếu mỗi thừa số hơi nhỏ hơn 1 thì sau vài chục khối nó tắt — đúng cơ chế ở C/12/02. Ở khối residual, đường cộng thêm một số hạng I vào Jacobian, và số hạng đó không phụ thuộc trọng số nên không thể bị nhân cho tiêu biến. Đây là lý do bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải sai số kiểm thử: thứ được sửa là khả năng tối ưu hoá, không phải sức chứa.

Hình 4 cũng cho thấy cái giá đã nhắc ở bảng: mũi tên đường tắt buộc tensor x phải sống tới tận điểm cộng, nên bộ nhớ kích hoạt của một mạng residual cao hơn một mạng phẳng cùng độ sâu.

Cái giá thì cụ thể: nhánh tắt buộc giữ tensor x sống tới tận chỗ cộng, nên bộ nhớ kích hoạt tăng so với một mạng phẳng cùng độ sâu.

↪ cùng nguyên lý với C/12/02 — kết nối tắt, hàm kích hoạt, khởi tạo và chuẩn hoá đều nhắm vào một đại lượng: hệ số nhân của tích các Jacobian

E. Các biến thể convolution và mục đích riêng

Đây là một bộ sưu tập chứ không phải một chuỗi, nên đọc theo kiểu tra cứu. Điểm chung: mỗi biến thể *nói lỏng hoặc siết chặt* đúng một trong ba giả định gốc ở mục A.

- **Depthwise separable** — tách tương tác không gian khỏi tương tác kênh.
 - **Được:** chi phí giảm khoảng k^2 lần, như phép tính ở mục C.
 - **Mất:** cường độ số học tụt nên độ trễ thật giảm ít hơn nhiều; và giả định “không gian và kênh tách rời được” không phải lúc nào cũng đúng.
- **Group convolution** — chia kênh thành g nhóm, mỗi nhóm chỉ nói chuyện trong nội bộ. Ra đời từ ràng buộc bộ nhớ của AlexNet trên hai GPU, sau đó thành công cụ giảm tham số. **Mất:** thông tin không đi qua ranh giới nhóm, nên cần một phép trộn kênh (ví dụ shuffle hoặc 1×1) để bù.

- **Dilated convolution** [12] — giãn khoảng cách giữa các điểm lấy mẫu trong cửa sổ, nên trường tiếp nhận tăng theo cấp số nhân mà không cần giảm độ phân giải. Sinh ra cho segmentation, nơi giảm độ phân giải là mất mát trực tiếp. *Mất*: hiện tượng lưới — các ô lấy mẫu không phủ kín nên xuất hiện hoa văn răng lược nếu xếp chồng nhiều tầng cùng hệ số giãn.
- **Deformable convolution** [13] — học luôn cả *vị trí* lấy mẫu thay vì cố định trên lưới, tức nói lỏng giả định cục bộ theo một cách có điều khiển. *Mất*: truy cập bộ nhớ không đều nên rất khó tối ưu trên GPU, và là một trong những lý do phổ biến khiến một mô hình không xuất được sang runtime suy luận.
- **Transposed convolution** — tăng độ phân giải ở nhánh giải mã. *Mất*: hiện tượng bàn cờ khi kích thước kernel không chia hết cho stride; nhiều cài đặt hiện đại thay nó bằng *upsample rồi conv thường* để tránh hẳn.

F. ConvNeXt và bài học đắt nhất của cả mục

Khi ViT vượt CNN trên ImageNet, cách đọc tự nhiên là “*attention* tốt hơn convolution”. ConvNeXt kiểm chứng cách đọc đó bằng một thí nghiệm có kỷ luật. Nhóm tác giả giữ nguyên convolution, rồi lần lượt áp từng thành phần của recipe hiện đại và đo sau mỗi bước [11]. Danh sách gồm lịch học dài hơn, AdamW, augmentation mạnh, stochastic depth, LayerNorm thay Batch-Norm, kernel lớn hơn, và tỉ lệ tăng theo kiểu transformer. Kết quả là một CNN thuần bắt kịp ViT cùng hạng.

Kết luận đúng không phải “CNN thắng” mà là: *phần lớn khoảng cách được gán cho kiến trúc thực ra thuộc về công thức huấn luyện*. Điều này thay đổi cách bạn đọc mọi paper kiến trúc về sau. Câu hỏi đầu tiên khi thấy một bảng so sánh không còn là “kiến trúc nào cao hơn”. Nó thành “hai dòng này có cùng recipe không, và nếu không thì ai được tune kỹ hơn”. Đây cũng là lý do chương E/24-thi-nghiem-va-ablation tồn tại.

↪ **hệ quả của E/24** — một bảng so sánh chỉ có nghĩa khi hai dòng được huấn luyện bằng cùng công thức và được tune ngang nhau

G. Giới hạn và trường hợp hỏng

Ba giới hạn đáng mang theo. Thứ nhất, *FLOPs không phải độ trễ, và số tham số cũng không*. Ba đại lượng này tương quan yếu với nhau và với thời gian chạy thật; MobileNet và DenseNet là hai ví dụ ngược chiều nhau về đúng điểm này. Bất kỳ lựa chọn backbone nào cũng phải đo trên đúng phần cứng và đúng chế độ batch mà bạn sẽ triển khai.

Thứ hai, *độ chính xác ImageNet là một chỉ số đại diện, không phải mục tiêu*. Tương quan giữa nó và chất lượng ở tác vụ khác là có thật, nhưng không chặt. Tương quan đó yếu đi rõ rệt với các tác vụ dự đoán dày như segmentation. Ở đó, độ phân giải và đặc trưng đa tỉ lệ quan trọng hơn khả năng phân loại toàn ảnh.

Thứ ba, các con số trong mục này thuộc hai loại khác nhau. Các phép tính tham số và FLOPs ở mục C là số học thuần túy nên chính xác. Ngược lại, các phát biểu kiểu “rẻ hơn 8,7 lần nhưng độ trễ giảm ít hơn nhiều” là mô tả định tính của một hiện tượng đã được quan sát rộng rãi; mức giảm cụ thể phụ thuộc GPU, thư viện và kích thước tensor tới mức tôi không nêu một con số nào ở đây.

CẠM BẤY

Cạm bẫy phổ biến nhất khi chọn backbone là so sánh hai mô hình được huấn luyện bằng hai recipe khác nhau rồi kết luận về kiến trúc. Một ResNet-50 theo recipe 2015 và một ResNet-50 theo recipe 2022 chênh nhau vài điểm phần trăm trên ImageNet. Khoảng cách

đó lớn hơn khoảng cách giữa nhiều cặp kiến trúc mà người ta viết hẳn một paper để so. Trước khi tin bất kỳ bảng nào, hãy kiểm tra ba thứ: số epoch, bộ augmentation, và optimizer cùng lịch học.

CẦN NHỚ

Mỗi bước trong dòng CNN gỡ đúng một nút thắt và đẩy độ khó sang một trục khác: tham số $\rightarrow$ khả năng tối ưu hoá $\rightarrow$ chi phí tính toán $\rightarrow$ độ trễ trên phần cứng thật $\rightarrow$ công thức huấn luyện. Khi cầm một backbone mới, hỏi nó đang đứng ở nút nào trong chuỗi đó — câu trả lời cho bạn biết nó sẽ hỏng ở đâu. [T2]

H. Kết nối

Mục này giả định toàn bộ C/12-co-che-huan-luyen, và trả lại cho chương đó thành viên thứ tư của họ “giữ tích Jacobian gần 1”. Nó đi thẳng vào C/13/02-tu-chuoi-toi-attention, nơi câu hỏi “bao nhiêu giả định là đủ” được đặt cho dữ liệu có thứ tự. Rồi vào C/13/03-vit-va-inductive-bias, nơi ba giả định ở mục A bị gỡ bỏ có chủ đích để xem chuyện gì xảy ra. Phần FLOPs và cường độ số học nối trực tiếp sang B/11-gioi-han-tinh-toan và D/20-toi-uu-va-nen-mo-hinh; còn bài học ConvNeXt là tiền đề của E/24-thi-nghiem-va-ablation. Nếu bạn quan tâm việc lắp backbone vào một hệ thống thật, C/14-bai-toan-cv-loi là chương nói về phần neck và head gắn lên nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba giả định mà convolution cài sẵn là gì, và mỗi biến thể ở mục E nói lỏng giả định nào?
2. Vì sao xếp chồng hai tầng 3×3 tốt hơn một tầng 5×5 ở cả ba mặt, và trong tình huống nào nó không tốt hơn?
3. Bằng chứng gốc của ResNet là sai số *huấn luyện* chứ không phải kiểm thử. Vì sao chi tiết đó thay đổi hoàn toàn kết luận rút ra?
4. MobileNet giảm FLOPs khoảng 8,7 lần nhưng độ trễ giảm ít hơn nhiều — giải thích bằng cường độ số học thì thế nào, và phép đo nào xác nhận chẩn đoán đó?
5. Nếu ConvNeXt đúng khi nói recipe quan trọng hơn kiến trúc, điều đó thay đổi thứ tự bạn đọc một paper kiến trúc thế nào?
6. Deformable convolution mạnh hơn conv thường ở đâu, và vì sao nó thường bị loại khi triển khai lên thiết bị nhúng?

Nguồn

- [1] Y. LeCun, L. Bottou, Y. Bengio, et al., “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *NeurIPS*, 2012.
- [3] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *ICLR*, 2015.
- [4] C. Szegedy, W. Liu, et al., “Going Deeper with Convolutions,” in *CVPR*, 2015.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [6] K. He, X. Zhang, S. Ren, et al., “Deep Residual Learning for Image Recognition,” in *CVPR*, 2016.
- [7] G. Huang, Z. Liu, L. van der Maaten, et al., “Densely Connected Convolutional Networks,” in *CVPR*, 2017.
- [8] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [9] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *CVPR*, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [11] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [12] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” in *ICLR*, 2016.
- [13] J. Dai, H. Qi, et al., “Deformable Convolutional Networks,” in *ICCV*, 2017.